1 Мета роботи

Дослідити захист ключів і побічні канали. Запропонувати керування секретами backend і БД та перевірити принципи прикладами Python.

2 Зберігання та використання ключів

1. Код і файли витікають через Git, копії та надмірні права. Пароль поряд із зашифрованим ключем не дає належного розділення.

2. Keychain/Keystore контролюють доступ до секретів; апаратна ізоляція залежить від платформи й типу ключа.

3. TPM захищає ключі й може пов’язувати доступ зі станом платформи; застосування - диск, автентифікація, атестація.

4. Secure Enclave - ізольована підсистема Apple для захисту даних та операцій без видачі апаратних ключів застосунку.

5. HSM зберігає ключі й виконує операції в захищеному модулі; застосування - критичний серверний підпис.

6. Файл залежить від прав ОС; Keychain - локальне сховище; TPM/ Enclave - ключі пристрою; HSM - критичні серверні ключі.

7. Exportable дозволяє виведення за політикою, non-exportable забороняє витяг. Для критичного підпису експорт забороняють.

8. PKCS 11 уніфікує сесії, ключові об’єкти й операції криптографічних токенів.

9. Сервіс автентифікує сесію, знаходить handle ключа й викликає C_SignInit/C_Sign. HSM повертає тільки підпис.

10. Ротація обмежує строк використання. Для старих даних потрібні версії ключів і збереження доступу або перешифрування.

11. Backup потрібний для відновлення даних; його шифрують, обмежують доступ і тестують. Кожна копія додає ризик витоку.

12. FIPS 140-3 та ISO/IEC 19790 задають вимоги до модулів, ISO/IEC 24759 - випробування. Валідують конкретний модуль і конфігурацію.

3 Захист ключів в оперативній пам’яті

1. Під час програмної операції ключ із зашифрованого файла потрапляє до RAM і може бути доступним привілейованому спостерігачу.

2. Memory/core/crash dump зберігає пам’ять, swap виносить її на диск. Потрібні контроль дампів і захищений swap.

3. Очищення буфера скорочує час експозиції, але не усуває вже створені копії чи попередні знімки.

4. Автоматична пам’ять і копії Python ускладнюють стирання: del не гарантує перезапису. Перезапис bytearray не очищує незалежний bytes.

5. HSM виконує операцію без ключа в RAM застосунку; вхідні дані, результат і дозвіл виклику все одно захищають.

4 Побічні канали та перевірка MAC

Timing attack використовує час виконання; cache attack - сліди доступу до кешу; power analysis - споживання струму; electromagnetic analysis - електромагнітне випромінювання. Витік виникає через реалізацію та апаратне середовище, навіть за математично стійкого алгоритму.

```
for i in range(len(expected_mac)):
    if received_mac[i] != expected_mac[i]:
        return False
return True
```

Ранній вихід робить кількість кроків залежною від першої відмінності. За відповідних умов час може підказувати довжину правильного префікса MAC. Фрагмент також не перевіряє довжину: короткий ввід може спричинити помилку, а подовжений правильний префікс - бути прийнятим.

Constant-time comparison уникає раннього виходу залежно від вмісту. Використовують hmac.compare_digest і фіксований формат MAC. Однаковий час кожного запуску не гарантується; тип і довжина можуть бути видимі.

5 Аналіз шести ситуацій порушення безпеки

1. Ключ у коді Git. Доступ до репозиторію, історії чи збірки відкриває секрет. Прибирають ключ із коду, застосовують сховище секретів і службову ідентичність. Уже оприлюднений ключ замінюють; видалення рядка не відкликає копії.

2. Master key у конфігурації поруч із БД. Захист може спрацювати при викраденні лише зашифрованої БД, якщо ключ не потрапив до копії. Повна компрометація сервера чи спільний backup відкриває обидва. Доцільно відокремити KEK у KMS/HSM і видавати доступ за політикою.

3. Секрети в журналах. Ключі, токени та відкриті дані поширюються до лог-сховищ, операторів і резервних копій. Журналюють подію, key ID, час і результат, а не значення секрету; контролюють доступ і строк зберігання. Розкриті credentials відкликають, ключі замінюють.

4. Ключ підпису недоступний адміністратору сервера. Пару створюють у HSM як sensitive і non-extractable; сервіс працює через PKCS 11 або аналогічний API. Адміністрування HSM відділяють від хоста. Це ускладнює копіювання ключа, але захоплений сервіс може зловживати дозволеним підписом: потрібні авторизація, ліміти й аудит.

5. HMAC порівнюється звичайною функцією. Якщо реалізація завершує перевірку при першій відмінності, виникає потенційний часовий витік префікса. Застосовують compare_digest для значень очікуваного типу й довжини. Правильний HMAC не виправляє вразливу перевірку.

6. Production key видалено з публічного Git через кілька хвилин. Цього недостатньо: копії, кеші й історія могли зберегтися. Ключ вважають скомпрометованим, припиняють нове використання, відкликають пов’язані credentials/сертифікати, вводять новий ключ і досліджують наслідки. Для зашифрованих даних оцінюють потребу перешифрування. Очищення репозиторію доповнює, але не замінює ці дії.

6 Система керування ключами backend сервісу

Таблиця 1 - Розміщення секретів права доступу та ротація

Секрет	Призначен	Зберігання	Хто має	Експорт	Ротація
Ключ шифрування БД / DEK	AES-256-GCM для конфіденційних	У БД лише захищений DEK; KEK у KMS/HSM	Backend за службовою ідентичністю	DEK тимчасово в RAM;	Нові DEK для нових даних; KEK - з повторним
Ключ цифрового підпису	Підпис окремих документів	HSM; окрема пара для підпису	Сервіс підпису; адміністратори керують політикою	Ні; резервування лише захищеним	Нова пара і key ID; старий відкритий ключ для перевірки
TLS private key	Автентифікація TLS шлюзу	OS Keystore шлюзу або HSM із підтримкою	TLS шлюз; окремий сервіс поновлення	За політикою - ні	Нова пара і сертифікат до завершення строку дії
API credential	Доступ до зовнішнього API	Сховище секретів; мінімальний	Лише потрібний backend	Тимчасово видаєтьс	Короткий TTL або ротація з перекриттям і

При компрометації DEK: новий ключ і перешифрування уражених даних. При витоку ключа підпису: припинення підписання, повідомлення перевіряльників і нова пара. Для TLS: перевипуск та відкликання сертифіката. Для API credential: негайне відкликання у провайдера, новий обмежений credential та перевірка дій.

6.1 Схема компонентів та межі доступу

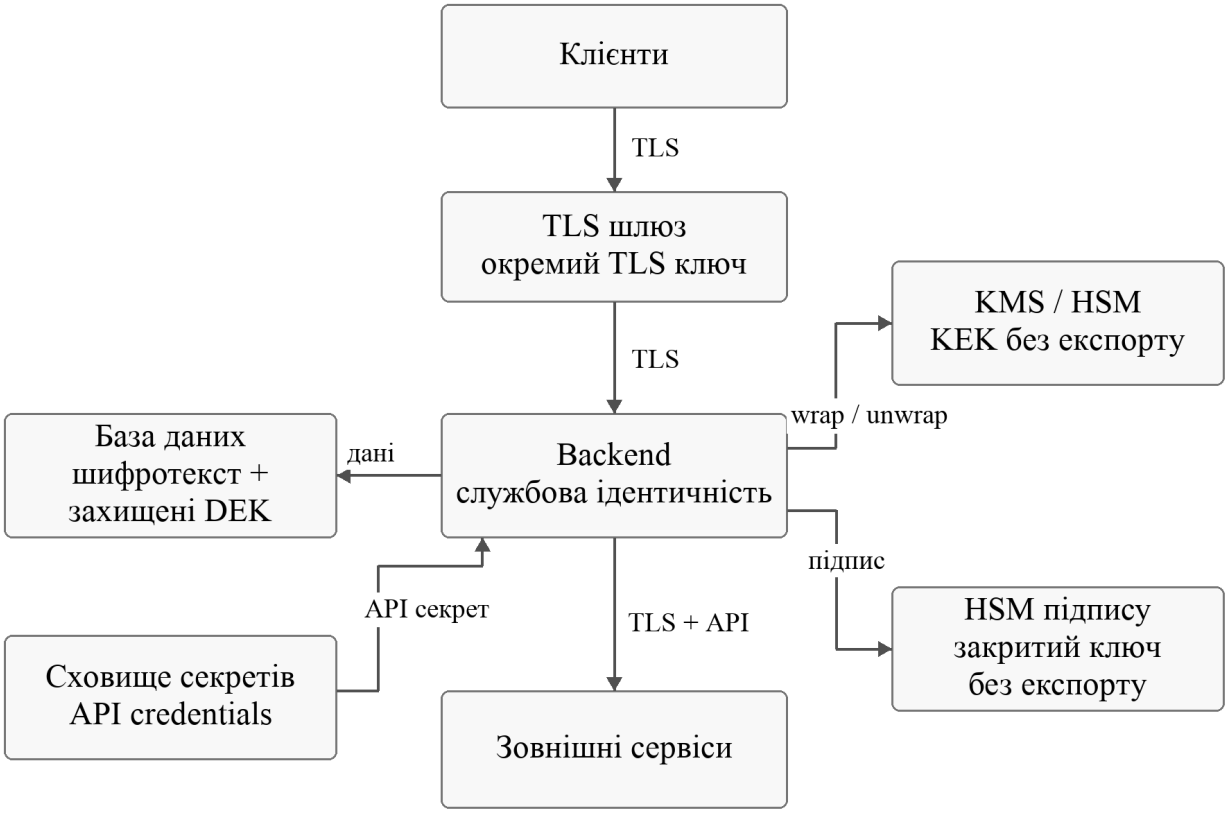


Рисунок 1 - Запропонована архітектура керування ключами

Backend шифрує записи DEK. БД зберігає шифротекст, nonce, тег, захищений DEK і версію KEK. KEK міститься в KMS/HSM; право unwrap прив’язане до службової ідентичності.

Підпис, TLS і шифрування мають окремі ключі. API credentials надходять зі сховища з мінімальними правами. Зв’язки з БД, модулями й зовнішніми API захищають та автентифікують.

Спільне сховище ключів, даних і коду створює одну точку компрометації. Розділення обмежує наслідки викрадення БД, але не приховує відкриті дані від захопленого авторизованого backend.

Передбачено захищені backup і перевірку відновлення. Для RAM обмежують час життя DEK, дампи та swap. Аудит містить суб’єкта, операцію й key ID без значень секретів.

7 Навчальні приклади Python

Python 3.12.14, cryptography 50.0.2, macOS ARM64. LocalKMS - модель у тому самому процесі без апаратної ізоляції та автентифікації. Приклад перевірки MAC:

```
return len(received) == len(expected) and \
    hmac.compare_digest(received, expected)
```

```
(.venv) (venv) ~/PycharmProjects/DKZ/lab4_project % python lab4.py all
ЗАПУСК: python lab4.py comparison
HMAC-SHA256: MAC має 32 байти
compare_digest: правильний MAC: ПРИЙНЯТО
Змінений / короткий / подовжений MAC: ВІДХИЛЕНО
Наївний цикл: позиції першої відмінності 0, 15, 31
Наївний цикл: кількість порівнянь [1, 16, 32]
Фрагмент без перевірки довжини помилково прийняв MAC + 1 байт.
Це підрахунок кроків, а не вимірювання часу чи timing attack.
Очікувані результати: 5/5
```

```
ЗАПУСК: python lab4.py memory
Пам'ять: навчальний bytearray має 32 байти
Початковий буфер після перезапису: НУЛІ
Незалежна bytes-копія зберегла початкові дані: ТАК
Очищення всіх копій у процесі Python цим не гарантовано.
Очікувані результати: 2/2
```

Рисунок 2 - Скриншоти перевірки MAC та буфера пам’яті

```
ЗАПУСК: python lab4.py rotation
Envelope encryption: AES-256-GCM для даних і захисту DEK
LocalKMS: модель в одному процесі, БЕЗ апаратної ізоляції
Версії KEK: kek-v1 -> kek-v2; DEK лишився тим самим
Розшифрування до ротації: ОК
Розшифрування після ротації та вимкнення v1: ОК
Шифротекст даних після зміни обгортки: БЕЗ ЗМІН
Стара вимкнена версія: ВІДХИЛЕНО (PermissionError)
Підміна даних / контексту / обгортки: ВІДХИЛЕНО (InvalidTag)
Сирі DEK і KEK не записано у файли та не виведено.
При витоку DEK потрібен новий DEK і перешифрування даних.
Очікувані результати: 7/7

(.venv) (venv) ~/PycharmProjects/DKZ/lab4_project %
```

Рисунок 3 - Скриншот виконання моделі ротації KEK

Усі 14 перевірок успішні: MAC - 5/5, пам’ять - 2/2, ротація - 7/7. Ключі не експортовано; повне очищення RAM не гарантовано.

8 Життєвий цикл двох типів ключів

У таблиці для кожного етапу наведено загрозу, компонент із доступом, захід захисту та дію при інциденті. KEK доступний лише KMS/HSM; відкритий DEK тимчасово отримує дозволений backend.

Таблиця 2 - Життєвий цикл DEK і ключа підпису

Етап	DEK шифрування даних	Закритий ключ підпису
Генерація	Слабка ентропія; backend. CSPRNG та ID. Дефект - новий DEK і перешифрування.	Копія поза модулем; роль HSM. Створення без експорту. Підозра - нова пара.
Зберігання	Викрадення БД; backend через KMS. Лише захищений DEK. Витік - перевірити DEK і KEK.	Надмірні права; HSM і керівник політик. Захищене резервування. Витік - ізоляція ключа.
Використання	RAM і логи; обробник запису. Короткий час життя, AEAD, унікальні nonce. Витік - заміна DEK.	Незаконний підпис; сервіс підпису. Авторизація, ліміти, аудит. Інцидент - блокування доступу.
Ротація	Втрата доступу; backend і KMS. Версії та перевірка міграції. Витік DEK - перешифрування.	Плутанина версій; керівник ключів і сервіс. Нова пара та ID. При витоку - екстрений перехід.
Відкликання та знищення	Залишкові копії; керівник ключів. Блокування й облік backup. Знищення після завершення потреби.	Подальша довіра; роль HSM. Зупинка підпису, сповіщення й відкликання сертифіката; знищення за політикою.

Планова ротація має підготовлену міграцію; екстрена заміна починається з локалізації витоку й відкликання. Rewrap при зміні KEK не усуває витоку DEK. Якщо KEK і старі обгортки були доступні нападнику, може знадобитися новий DEK та перешифрування.

9 Висновки

Ключі потребують розділених сховищ і прав, захисту RAM та керованого життєвого циклу. Запропоновано архітектуру БД, підпису, TLS й API; усі 14 перевірок успішні.